From Chaos to Cohesion

Building Modular Monoliths in the Real World

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com



Abstract

From Chaos to Cohesion: Building Modular Monoliths in the Real World

Many teams want better structure in their applications, but real projects come with history. Boundaries are unclear, responsibilities overlap and changes become risky. Spring Modulith offers good tools, but applying modularity in an existing codebase is a learning process, not a switch.

In this session, we walk through a practical approach to improving an existing monolith step by step. We look at how to find boundaries, introduce modules safely and use tests and documentation to keep the structure clear. The focus is on realistic progress inside a working product, not on a full rewrite.

After the talk, you will have a better idea how to start and how to continue this journey in your own environment.


helvetia
KARAKUN
Kanton Basel-Stadt
ERNI
INNOFUSE
+++Ba
sel++
++Te+
+++ch
Basel
basel.ch
JAVA
USER
GROUP
CH
BLKB
optravis
company of .msg
cyon
CS2
CSS
IRIX
swiss made
software
ICT
Scouts
Wir fördern Zukunft
SD:>_
[SwissDevJobs.ch]
#BaselOne26
baselone.ch



From Chaos to Cohesion

Building Modular Monoliths in the Real World

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com

Who Am I?



Patrick Baumgartner

Technical Agile Coach @ **42talents**

Focus on the **development of software solutions with humans**

Coaching, Architecture, Development, Reviews, Training

Lecturer @ **Zurich University of Applied Sciences ZHAW**

Co-Organizer of **Voxxed Days Zurich** and **JUG Switzerland**, Java Champion, Oracle ACE Pro Java, ...

Agenda

Agenda

1. **The Problem** | Why structure erodes in real codebases
2. **The Vision** | What "modular" actually means
3. **The Journey** | 7 steps from mess to structure (with live demo)
4. **Going Further** | Event externalization & when to stop
5. **Monday Morning** | How to start in your own project

ACT 1

The Problem: Why Structure Erodes

Who Has Felt Lost in a Codebase?

The Hidden Cost of Tangled Code

- Stripe 2018 study: developers worldwide waste **$85 billion/year**
- **~9%** of development time lost just reasoning about bad code
- This is not about bad developers, it is about eroding structure
- Every team has been there: the feature that touches 15 packages

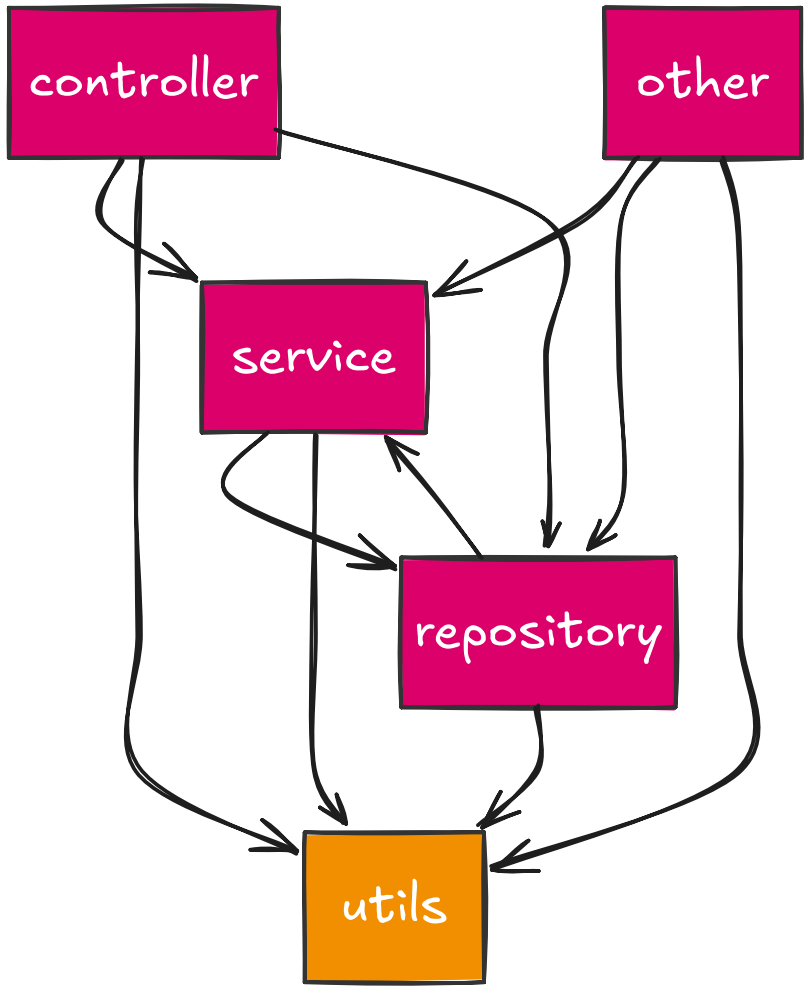
> *"We didn't plan this. Nobody planned this."*

How Monoliths Decay

Clean → Growing → Tangled → Big Ball of Mud

1. **Clean start**: Small team, clear packages
2. **Features grow**: New requirements, new devs, pressure to ship
3. **Boundaries blur**: "I'll just import it directly" - It's faster!
4. **Big ball of mud**: Every change is risky

The problem is not the monolith, it is the **missing structure**.



A Story From the Field

From Layers to Modules in a Living Product

- **80k LOC**, **4 teams**, **5 years** of history → classic package-by-layer
- First `verify()` : **87 violations**, **4 cycles**, one god-utils package
- First module cleaned up in **2 days**
- **4 sprints** later: 5 clean modules, zero violations, CI-enforced
- No rewrite. No feature freeze. **Shipped features during the migration.**

> *"The scariest part was running the test. After that, it was just work."*

Microservices Are Not Free

The Wrong Reaction to the Right Problem

- Network latency, partial failures, eventual consistency
- Operational complexity: deployments, observability, debugging
- Organizational overhead: team boundaries, API contracts, versioning
- Many teams adopt microservices **before they need them**

> The answer to a messy monolith is not distribution, it is **structure**.

The Horizontal Trap

Package-by-Layer Scatters Related Code

```
com.patbaumgartner.demo.layers
├── controller/
│   ├── OrderController
│   ├── InventoryController
│   └── CustomerController
├── service/
│   ├── OrderService
│   └── InventoryService
├── repository/
│   ├── OrderRepository
│   └── StockRepository
└── model/
```

```
com.patbaumgartner.demo.modulith
├── order/                          ← module
│   ├── OrderController
│   ├── OrderService
│   └── internal/
├── inventory/                      ← module
│   ├── ...
│   └── internal/
└── customer/                       ← module
```

Horizontal (left): related code is scattered. **Vertical** (right): related code lives together.

Modularity Is Not New

We Have Known This Since 1972

- **David Parnas, 1972**: *"On the criteria to be used in decomposing systems into modules"*
- **Tom DeMarco, 1979**: Structured Analysis and System Specification: Adopted cohesion and coupling
- **Simon Brown, 2015**: *"If you can't build a structured monolith, what makes you think microservices is the answer?"*
- **Kevlin Henney, 2024**: *"The missing word isn't 'monolith' — it's 'messy'."*

The tools are new. The principles are not. Spring Modulith gives us a way to **enforce** what we always knew.

ACT 2

The Vision: What "Modular" Means

What Is Spring Modulith?

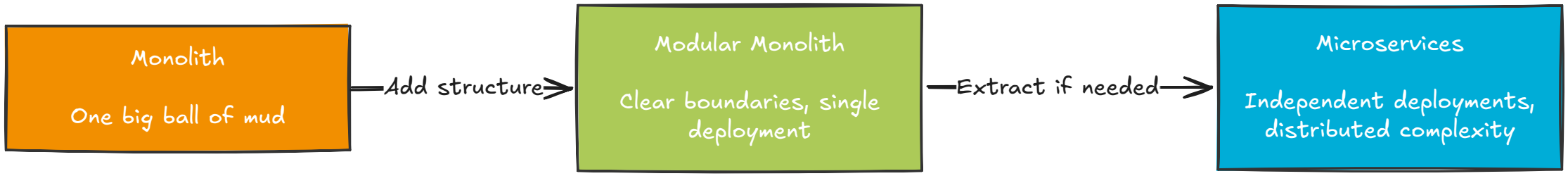
Structure Meets Framework

- A Spring Boot extension for building **well-structured monoliths**
- Provides conventions for module boundaries based on package structure
- Offers verification, testing, documentation, and runtime support
- Not a framework rewrite: **works with existing Spring Boot applications**



The Architecture Spectrum

From Monolith to Microservices



We are here: **Modular Monolith** → A single deployment, clear internal structure, real boundaries.

When to Choose a Modular Monolith

Building Modular Monoliths in the Real World

- Team size is small to medium
- Domain boundaries are still evolving
- Deployment simplicity matters
- You want structure without the distributed systems tax
- You are improving an **existing codebase**, not starting from scratch

> This is what "building modular monoliths in the real world" means: **practical structure inside a living product**.

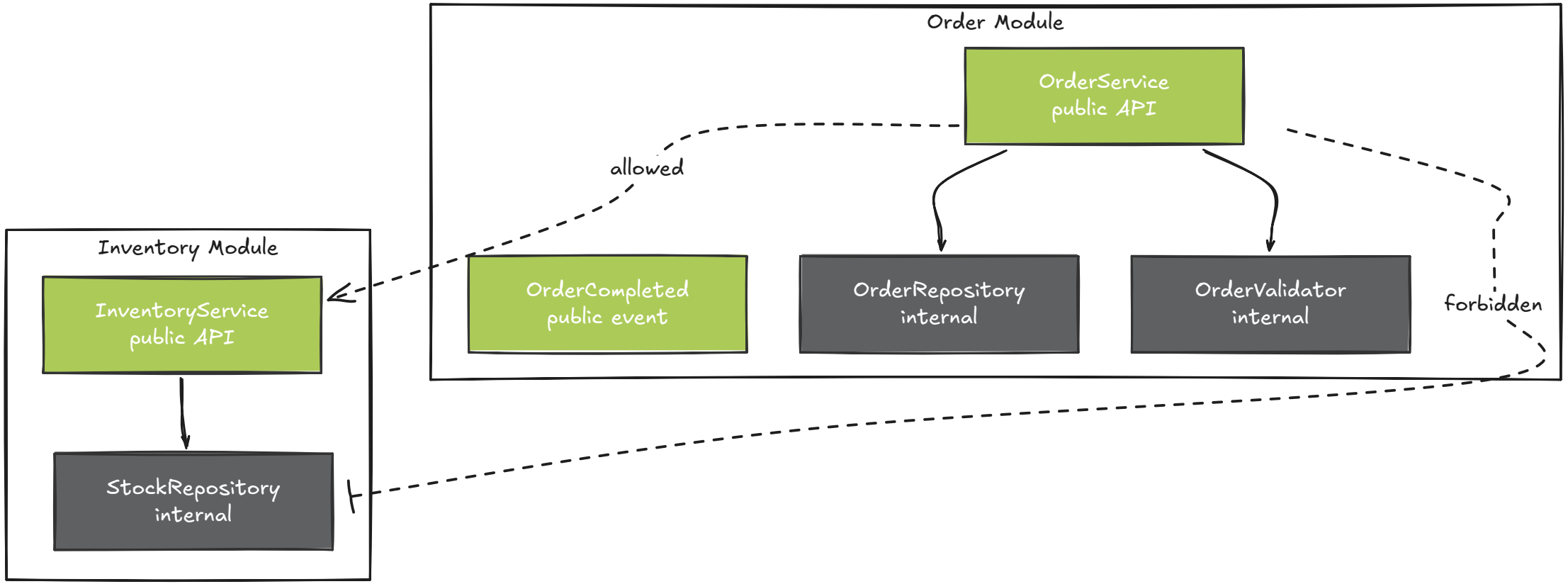
Vertical Slicing

Group by Domain, Not by Layer

- A **module** = API + internals + dependencies, grouped by business capability
- Spring Modulith's definition:
  - **Provided interface**: Spring bean instances exposed via API packages + published events
  - **Internal**: implementation components other modules must not access
  - **Required interface**: bean dependencies, listened events, configuration properties
- Top-level packages under `@SpringBootApplication` = modules
- Sub-packages = internal (not accessible from other modules)

Module Boundaries Visualized

Public API and Hidden Internals



Advanced Module Controls

When Conventions Are Not Enough

```
// Restrict allowed dependencies
@ApplicationModule(allowedDependencies = { "customer", "inventory" })
package com.patbaumgartner.demo.modulith.order;
```

```
// Named interfaces for fine-grained access
@NamedInterface("spi")
package com.patbaumgartner.demo.modulith.order.spi;
```

```
// Open module: All types accessible (use sparingly!)
@ApplicationModule(type = ApplicationModule.Type.OPEN)
package com.patbaumgartner.demo.modulith.shared;
```

Dealing with Shared Code

The Shared Kernel

- Some code genuinely belongs to multiple modules: value objects, utility types
- Create a `shared` or `common` module for truly cross-cutting concerns
- Keep it **small and stable**, avoid a new god package
- Prefer **duplication over wrong abstraction** when unsure

ACT 3

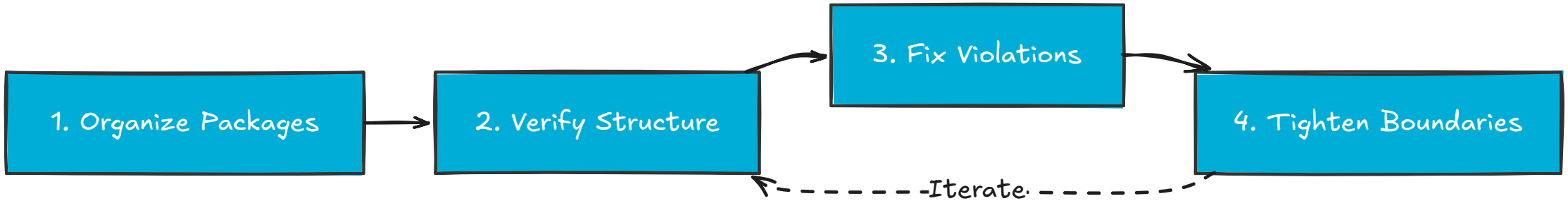
The Journey: 7 Steps to Structure

"It's a learning process, not a switch."

Do Not Rewrite

Incremental Improvement

- Restructure one area at a time, keep the app deployable after every step
- Use version control to track and reverse changes if needed



Finding Boundaries First

Assess Before You Restructure

- **Follow the domain**: align modules with business capabilities
- **Follow the data**: group code around the entities it owns
- **Follow the team**: modules should map to team responsibilities
- Talk to domain experts and read existing documentation
- Use **jdepend**, **ArchUnit**, or **Spring Modulith** verification to visualize dependencies

Step 1 | The Working Mess

This Compiles. This Ships. This Is Debt.

```java
@Service
class OrderService {

    // ⚠️ Violation! Crosses module boundary into inventory.internal
    private final StockRepository stockRepository;

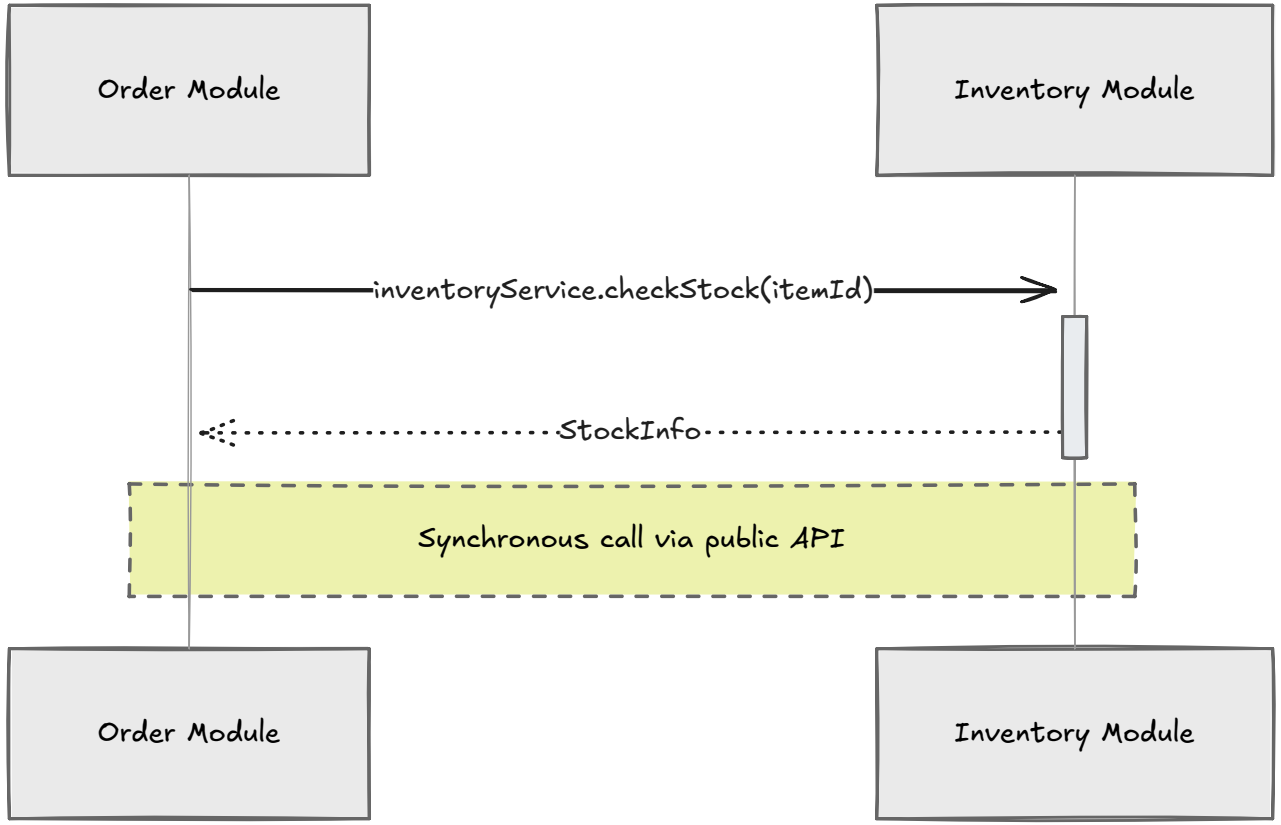
    String createOrder(String productId) {
        Stock stock = stockRepository.findByProductId(productId)
            .orElseThrow(() -> new IllegalStateException("Product not in stock: " + productId));
        stock.setQuantity(stock.getQuantity() - 1);
        stockRepository.save(stock);
        // ...
    }
}
```

`OrderService` directly imports `StockRepository` from `inventory.internal`.

This is **architectural debt**!

Direct API Calls

The Coupling We Need to Break



Step 2 | The Whistleblower

One Test to See the Truth

```
class ArchitectureTests {

    @Test
    void verifyModularStructure() {
        ApplicationModules.of(ModulithApplication.class).verify();
    }
}
```

Add `spring-modulith-starter-test`. Write this test. **Run it.**

Step 2 | The Whistleblower

🔴 RED | The Structure Is Broken

```
Module 'order' depends on non-exposed type
  com.patbaumgartner.demo.modulith.inventory.internal.StockRepository
  within module 'inventory'!
```

The test catches what the compiler cannot:

- **Illegal cross-module access** to internal types
- **Cyclic dependencies** between modules
- **Undeclared dependencies** when using `allowedDependencies`

> *"One test. That's all it takes to see the truth."*



🔴 → 🟢 LIVE DEMO

Step 2: verify() → RED | Step 3: Refactor → GREEN

Let's have a look at the code...

Step 3 | Decoupling with Events

Replace Direct Access with an Event

```
// order module: publishes event, no import from inventory
public record OrderCompleted(String orderId, String productId) {}

@Service
class OrderService {
    private final ApplicationEventPublisher events;
    String createOrder(String productId) {
        events.publishEvent(new OrderCompleted(orderId, productId));
        return orderId;
    }
}
```

```
// inventory module: listens, no dependency on order internals
@Component
class InventoryListener {
    @ApplicationModuleListener
    void on(OrderCompleted event) { /* decrease stock */ }
}
```

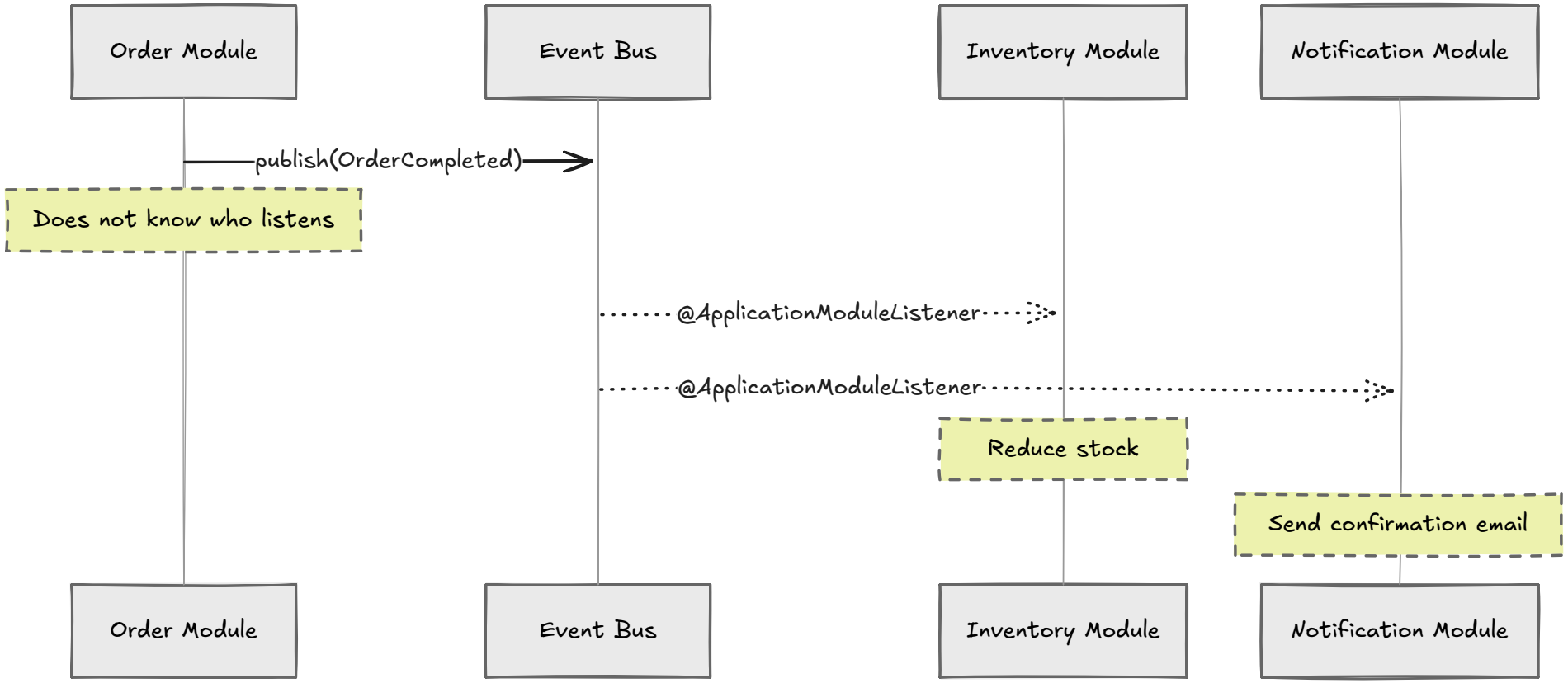
Step 3 | Result

🟢 GREEN | verify() Passes

- `OrderService` no longer imports anything from `inventory.internal`
- The event `OrderCompleted` is part of the **order** module's public API
- `InventoryListener` subscribes via `@ApplicationModuleListener` :
  - `@TransactionalEventListener + @Async + @Transactional` combined in one annotation
  - Listener runs in its **own transaction**
- **We went from spaghetti to events in one refactoring.**

Listening to Events

React Without Coupling



Breaking Dependency Patterns

Three Strategies

Pattern	When to Use	Example
Move the code	Type belongs in the other module	Move shared entity to owning module
Publish an event	Notification, no return value needed	`OrderCompleted` → `InventoryListener`
Invert the dependency	Extract interface into API package	`OrderService` calls `InventoryPort`

The Sisyphus Problem

Gradual Adoption in a Living Codebase

While you fix module A, new code in module B introduces violations.

- **`Type.OPEN`** on legacy modules, opt out temporarily
- **`explicitly-annotated` detection**, annotated packages only
- **Custom detection strategy**, grow the module set over sprints
- **Run `verify()` in CI**, catch regressions before merge

> Modularity is not a one-time effort. It is an **ongoing discipline**.

Step 4 | Living Documentation

Documentation That Cannot Drift

```
class ArchitectureTests {
    @Test
    void verifyAndDocument() {
        ApplicationModules modules = ApplicationModules.of(ModulithApplication.class);
        modules.verify();
        new Documenter(modules).writeDocumentation();
    }
}
```

Generates:

- **PlantUML C4** component diagrams of module dependencies
- **Module Canvases**: base package, Spring beans grouped by stereotype, aggregate roots, published & listened events, configuration properties
- Run in CI → diagrams always reflect the **actual** structure

Module Canvas

Document Module Responsibilities

```
new Documenter(modules)
  .writeModuleCanvases();
```

Generates a structured overview per module:

- **Base package**
- **Spring beans** exposed by the module, grouped by stereotype (Controllers, Services, Repositories, Event listeners, …)
- **Exposed aggregate roots**, **published & listened events**
- **Configuration properties** (needs `spring-boot-configuration-processor`)

Step 5 | Runtime Inspector

See Module Structure at Runtime

```
management.endpoints.web.exposure.include=modulith,health
```

```
GET /actuator/modulith
{
  "order": {
    "displayName": "Order",
    "basePackage": "com.patbaumgartner.demo.modulith.order",
    "type": "closed",
    "dependencies": []
  },
  "inventory": {
    "basePackage": "com.patbaumgartner.demo.modulith.inventory",
    "dependencies": [
      { "types": ["EVENT_LISTENER"], "target": "order" }
    ]
  }
}
```

Observe & Protect

Steps 6 & 7

Step 6 | Observability

Traces and the Event Publication Registry

- **OpenTelemetry + Jaeger**: Auto-creates spans for module interactions
- **Event Publication Registry**: Stores events in DB alongside business data

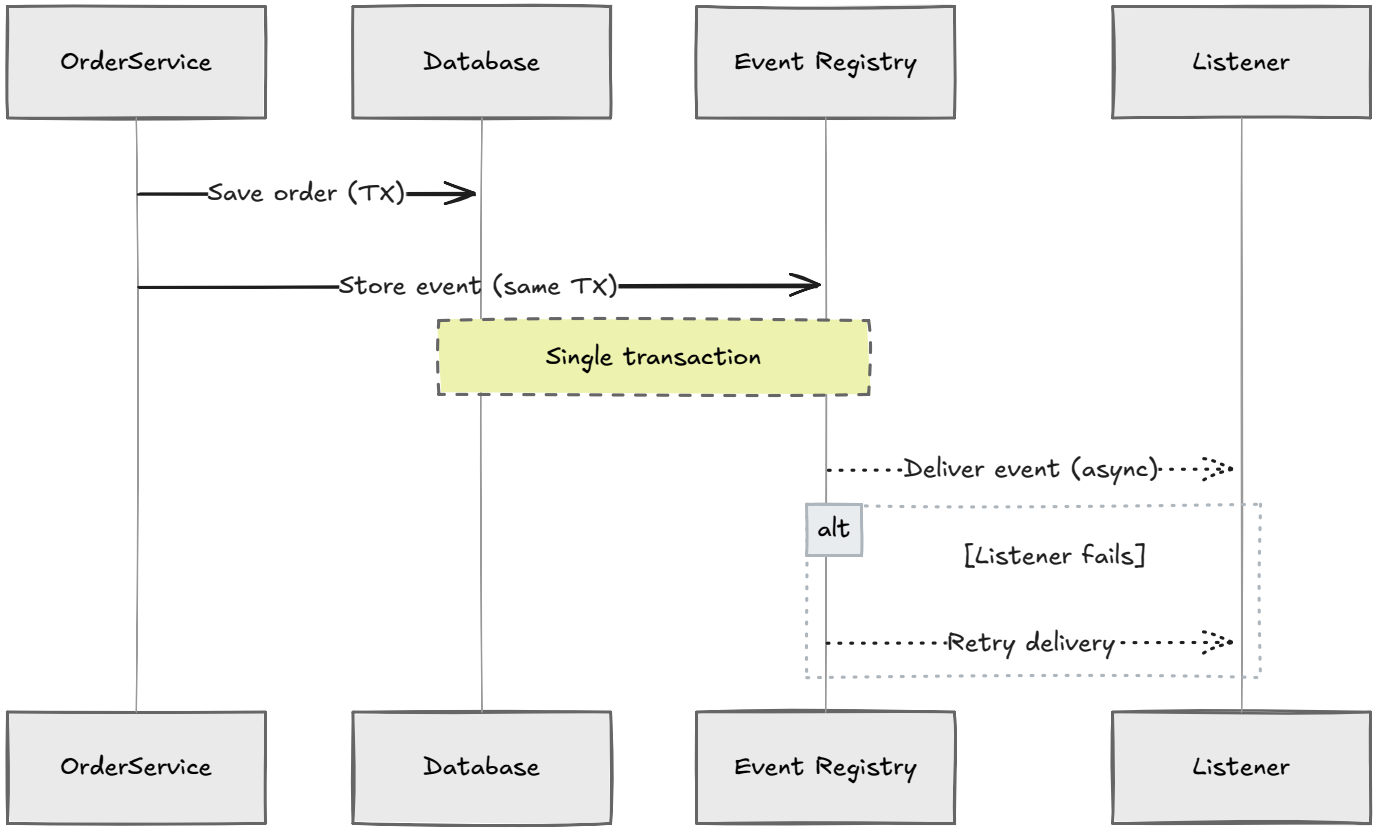
Add `spring-modulith-starter-jdbc` + `spring-modulith-starter-insight`.

```
@ApplicationModuleListener
void on(OrderCompleted event) {
    // Simulated failure!
    throw new RuntimeException("Inventory service temporarily unavailable!");
}
```

Result: `EVENT_PUBLICATION` table → `STATUS = 'FAILED'`. **Events survive crashes.**

Event Publication Registry

Reliable Event Delivery



Event Publication Lifecycle

Reliable Event Delivery (Spring Modulith 2.0+)

```
PUBLISHED → PROCESSING → COMPLETED ✓
                ↓
            FAILED → RESUBMITTED → PROCESSING → ...
```

- **Staleness monitor**: Marks stuck PUBLISHED / PROCESSING / RESUBMITTED events as FAILED
- **`ResubmissionOptions`** : Configure batch size, max in-flight, min age, filter
- **Completion modes**: `UPDATE` (default), `DELETE` , `ARCHIVE` , choose based on audit needs
- Supports **JDBC**, **JPA**, **MongoDB**, and **Neo4j**

Step 6 | Recovery

Fix the Listener, Restart, Events Get Processed

1. **Fix** the `InventoryListener`, remove the `throw`
2. **Restart** the application with `spring.modulith.events.republish-outstanding-events-on-restart=true`
3. Failed events are **resubmitted automatically**
4. Check `EVENT_PUBLICATION` table → `STATUS = 'COMPLETED'`

```
@ApplicationModuleListener
void on(OrderCompleted event) {
    // Fixed! Back to normal processing.
    Stock stock = stockRepository.findByProductId(event.productId()).orElseThrow();
    stock.setQuantity(stock.getQuantity() - 1);
    stockRepository.save(stock);
}
```

The safety net caught the failure. The recovery was **automatic**.

Step 7 | Scenario Testing

Test One Module. No Mocks for the Publisher Side.

```
@ApplicationModuleTest
class OrderIntegrationTests {

    @Autowired OrderService orderService;

    @Test
    void shouldPublishOrderCompletedEvent(Scenario scenario) {
        scenario.stimulate(() -> orderService.createOrder("PROD-001"))
            .andWaitForEventOfType(OrderCompleted.class)
            .matching(event -> "PROD-001".equals(event.productId()))
            .toArriveAndVerify(event ->
                assertThat(event.orderId()).isNotNull());
    }
}
```

- `@ApplicationModuleTest` bootstraps **only** the module + deps
- `Scenario` : stimulate → wait → verify

Integration Tests per Module

Test Modules in Isolation

```
@ApplicationModuleTest
class OrderModuleTests {

  @Test
  void bootstrapsOrderModule() {
    // Only beans from the order module and its
    // dependencies are loaded
  }
}
```

- Bootstraps only the module under test and its allowed dependencies
- Faster than loading the entire application context
- Verifies that the module can start independently

ACT 4

Going Further

Event Externalization

Publish Beyond the Monolith

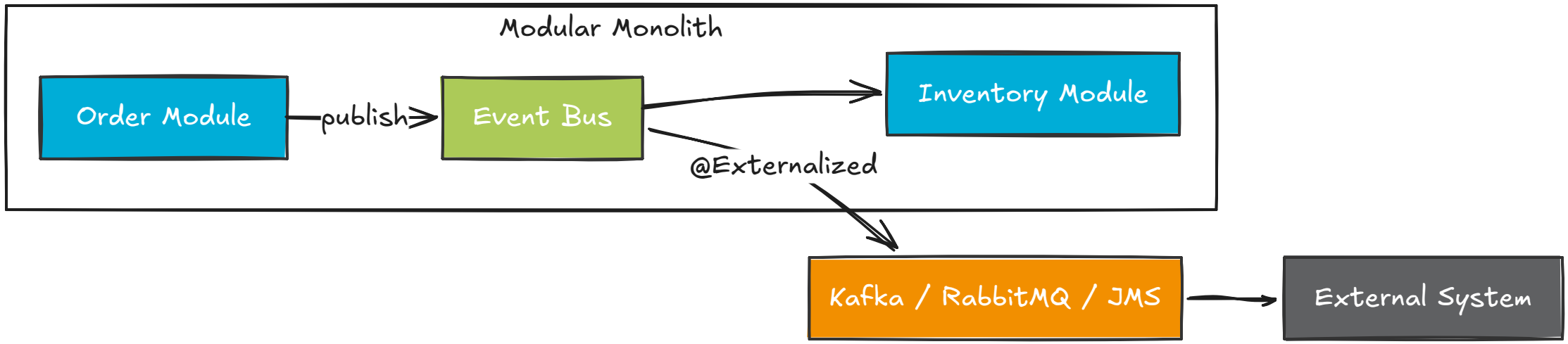
```
@Externalized("order-completed::#{#this.orderId}")
public record OrderCompleted(String orderId, String productId) {}
```

- Annotate events with `@Externalized` to publish to **Kafka**, **AMQP**, or **JMS**
- Same event contracts work **internally** and **externally**
- Transition path to microservices, if and when you need it

> *"Don't split for the sake of splitting."*

Event Externalization

Publish Beyond the Monolith



When NOT to Go Further

The Modular Monolith May Be Your Destination

- Not every monolith needs microservices
- A well-structured modulith may be the **right end state**
- Microservices add network latency, partial failures, operational complexity
- If your team is small, your domain fits in one deployment, and you ship fast, **stay modular**

> *"When you need microservices, you'll know. Until then, don't split."*

ACT 5

Monday Morning

How to Start Monday Morning

5 Concrete Steps

1. **Add the BOM** + `spring-modulith-starter-test` to your existing project
2. **Write the `verify()` test**, see what breaks
3. **Generate documentation**, see your actual architecture
4. **Pick one module**, refactor it, protect it with events
5. **Repeat**, grow the module set sprint by sprint

```
<dependency>
  <groupId>org.springframework.modulith</groupId>
  <artifactId>spring-modulith-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Q&A

Let's Talk About Your Context!

Key Takeaways

What to Remember

- **Modular monoliths** give you structure without the distributed systems tax
- **Spring Modulith** provides conventions, verification, testing, documentation, observability
- **Events** decouple modules and prepare for future extraction
- **Tests and CI verification** prevent structure regression
- **Documentation** keeps the architecture visible and shared
- **Start small**: one test, one module, one afternoon

Common Pitfalls

Raise Your Hand If...

- 🙋 ...you have a **"utils" or "common" package** that everything depends on
- 🙋 ...you **skipped a failing test** because "we'll fix it later"
- 🙋 ...you split into microservices and **wished you hadn't**
- 🙋 ...you started with **too many modules** and gave up

Every one of these is a trap. The antidote: **start small, enforce in CI, stay disciplined**.

Feedback is a Gift

Your turn 👇

👉 **Provide feedback!**

- 🐛 What didn't work (aka bugs)?
- 🤯 What was confusing (missing docs)?
- 🚀 What should I refactor next?

No feedback = it works on your machine™



From Chaos to Cohesion: Building Modular Monoliths in the Real World



From Chaos to Cohesion

Structure is a journey, not a destination.

You don't need permission to start. You don't need a rewrite.

> One module at a time. One test and one afternoon.


Thank You!
Let's Keep Building!



From Chaos to Cohesion

Building Modular Monoliths in the Real World

Patrick Baumgartner

42talents GmbH, Zürich, Switzerland

@patbaumgartner

patrick.baumgartner@42talents.com

Further Resources

Learn and Connect

- **Stripe Study 2018**: https://stripe.com/files/reports/the-developer-coefficient.pdf
- **David Parnas, 1972**: https://dl.acm.org/doi/10.1145/361598.361623
- **Simon Brown from ~2013**: https://simonbrown.je/modular-monolith
- **Spring Modulith**: https://spring.io/projects/spring-modulith
- **Spring Modulith Reference**: https://docs.spring.io/spring-modulith/reference/
- **Spring Boot**: https://spring.io/projects/spring-boot
- **ArchUnit**: https://www.archunit.org